

Chapter 06

Chains in LangChain

Learn how to connect prompts, models, and parsers into automated pipelines — and build Sequential, Parallel, and Conditional chains using LCEL.

WHAT THIS CHAPTER COVERS

- ▶ What chains are and why they exist
- ▶ LangChain Expression Language (LCEL)
- ▶ Building a Simple Chain
- ▶ Building a Sequential Chain
- ▶ Building a Parallel Chain with RunnableParallel
- ▶ Building a Conditional Chain with RunnableBranch
- ▶ Using RunnableLambda for default fallbacks
- ▶ Structured output for reliable branching logic

Chains in LangChain

RECAP — CHAPTER 4

The previous chapter covered how to generate structured, reliable output from an LLM using output parsers — the same structured-output pattern (Pydantic + parser) reappears in this chapter as the key to making Conditional Chains reliable.

1. What Are Chains and Why Do They Exist

Definition

A **Chain** in LangChain is a mechanism to connect multiple steps of an LLM-based application into a single automated pipeline, where the output of one step automatically becomes the input of the next step.

Why It Exists

Even a simple LLM application involves multiple discrete steps: taking a prompt from the user, sending it to the LLM, and processing the response before displaying it. Executing each of these steps manually — calling `.invoke()` on the prompt, then manually passing the result to the model, then manually passing that to the parser — becomes repetitive and hard to scale as applications grow more complex.

How It Works

Chains connect components (prompt, model, parser, and more) using the **pipe operator** (`|`). This declarative syntax is called **LangChain Expression Language (LCEL)**. Once a chain is built, only the first step needs an input — triggering (`invoke`) the chain automatically executes every step in sequence, with each step's output automatically routed as the next step's input.

IMPORTANT

Chains are not limited to linear/sequential pipelines. LangChain supports multiple pipeline structures: **Sequential** (linear, step-by-step), **Parallel** (multiple independent chains executed simultaneously), and **Conditional** (different chains executed based on a condition, like an if-else).

As shown in Figure 6.1 below, the general anatomy of a chain moves from raw user input, through formatting and model processing, to a clean final output.

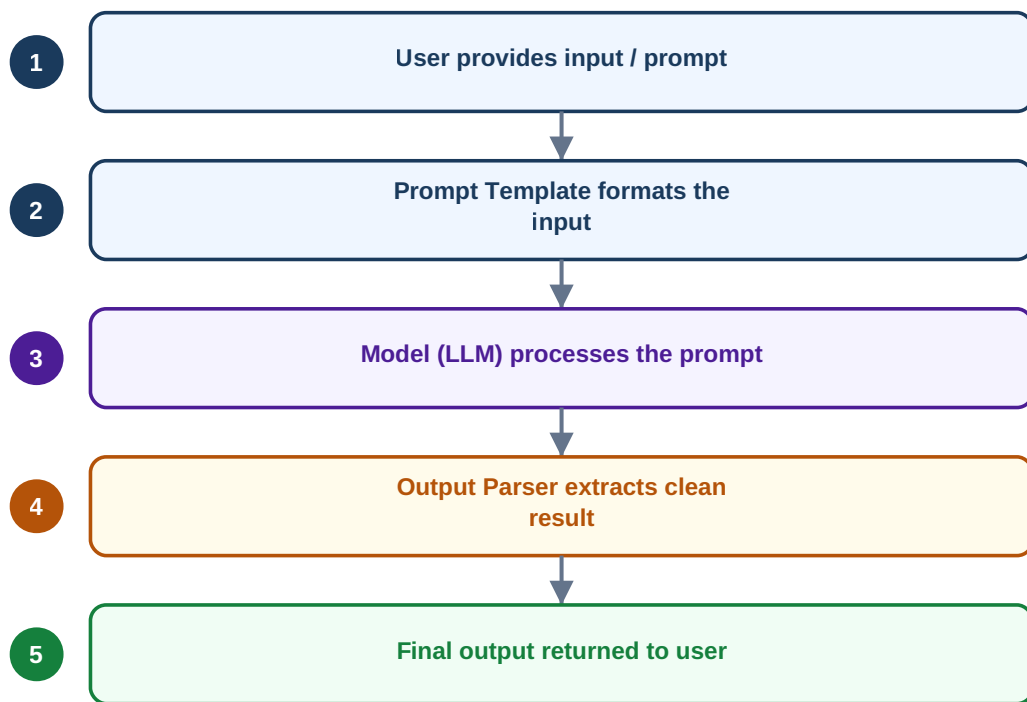


Figure 6.1 — General anatomy of a LangChain pipeline (chain).

🔄 Quick Revision — Chains: What & Why

- Chains connect multiple steps (prompt, model, parser, etc.) into one automated pipeline.
- Without chains, every component must be invoked manually and its output manually passed forward.
- The pipe operator `|` is used to build chains — this syntax is called LCEL (LangChain Expression Language).
- Three major chain structures exist: Sequential, Parallel, and Conditional.
- A chain's internal structure can be visualized using `chain.get_graph().print_ascii()`.

2. Simple Chain

Definition

A Simple Chain is the most basic chain type: **Prompt** → **Model** → **Parser**, executed once, in a straight line.

Why It Exists

It demonstrates the minimal chain pattern before moving to more complex structures such as a sequential chain with multiple LLM calls, a parallel chain, or a conditional chain.

How It Works

1. A `PromptTemplate` is built with an input variable.
2. A `ChatHuggingFace` model (wrapping a `HuggingFaceEndpoint`) is created.

3. A `StrOutputParser` converts the raw model response into a clean string.
4. These three are connected using the pipe operator into a single chain.
5. `chain.invoke({...})` triggers execution of the entire pipeline.

Example

```
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv

load_dotenv()

# Endpoint pointing to a hosted Qwen model on Hugging Face
llm = HuggingFaceEndpoint(
    repo_id = "Qwen/Qwen2.5-7B-Instruct"
)

# Wrap the endpoint in a chat-compatible interface
model = ChatHuggingFace(llm = llm)

prompt = PromptTemplate(
    template = "Generate 5 interesting facts about {topic}",
    input_variables = ["topic"]
)

parser = StrOutputParser()

# Build the chain using the pipe operator (LCEL)
chain = prompt | model | parser

result = chain.invoke({"topic": "Cricket"})

# Visualize the chain's internal structure
chain.get_graph().print_ascii()
```

The `PromptTemplate` defines the input variable `topic` and generates the actual prompt text once a value is supplied. `HuggingFaceEndpoint` connects to the `Qwen/Qwen2.5-7B-Instruct` model hosted on Hugging Face, and `ChatHuggingFace` wraps it so it behaves like a standard chat model within LangChain. The line `chain = prompt | model | parser` declares the full pipeline, and `chain.invoke(...)` triggers it with a single call — no manual intermediate invocations are needed.

INTERVIEW TIP

Be ready to explain that `chain.get_graph().print_ascii()` is a debugging/visualization utility that shows the exact sequence of `Runnables` in a chain — useful for demonstrating understanding of how LCEL pipelines are structured internally.

🔄 Quick Revision — Simple Chain

- Simple Chain = Prompt → Model → Parser.
- Built using the pipe operator: `chain = prompt | model | parser`.
- This project's implementations use `HuggingFaceEndpoint` + `ChatHuggingFace` instead of a direct OpenAI call.
- `StrOutputParser` extracts a clean string from the model's raw response object.
- `chain.get_graph().print_ascii()` visualizes the pipeline.

3. Sequential Chain

Definition

A Sequential Chain connects more than two steps in a straight line, where the LLM may be called more than once — the output of one LLM call becomes the input prompt for the next LLM call.

Why It Exists

Some tasks naturally require multiple stages of processing by the LLM. For example: first generate a detailed report on a topic, then summarize that report into key points. Each stage depends on the previous stage's output.

How It Works

1. **Prompt 1** asks the LLM to generate a detailed report on a topic.
2. The **model** processes Prompt 1 and returns a detailed report.
3. The **parser** extracts the report as a clean string.
4. This string becomes the input to **Prompt 2**, which asks the LLM to summarize the report into key points.
5. The **model** processes Prompt 2, and the **parser** extracts the final summary.

Figure 6.2 below shows both LLM calls chained end-to-end.

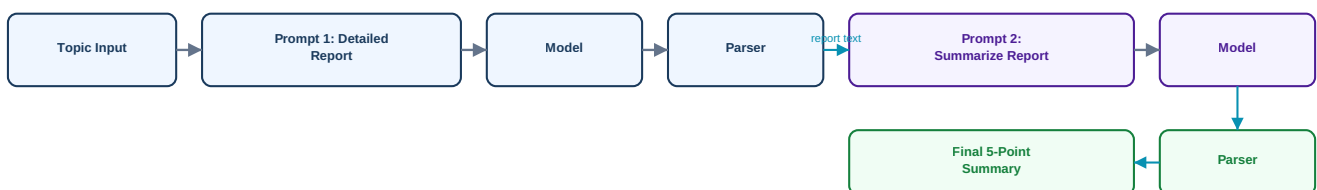


Figure 6.2 — Sequential Chain: report generation followed by summarization.

Example

```
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv

load_dotenv()

llm = HuggingFaceEndpoint(
    repo_id = "Qwen/Qwen2.5-7B-Instruct"
)

model = ChatHuggingFace(llm = llm)

# First stage: generate a detailed report on the given topic
prompt1 = PromptTemplate(
    template = "Generate a detailed report on the topic : {topic}.",
    input_variables = ["topic"]
)

parser = StrOutputParser()

# Second stage: summarize the report generated in stage one
prompt2 = PromptTemplate(
    template = "Generate a summary of 5 points based on the report \n {report}.",
    input_variables = ["report"]
)

# Chained twice through the same model – output of first parser feeds prompt2
chain = prompt1 | model | parser | prompt2 | model | parser

result = chain.invoke({"topic" : "Unemployment in India"})

chain.get_graph().print_ascii()
```

The chain is simply an extended pipe sequence. The output string of the first parser automatically satisfies the `{report}` input variable expected by `prompt2`, since the parser's string output becomes the sole input to the next Runnable. This pattern can be extended indefinitely to build arbitrarily long sequential chains.

BEST PRACTICE

Sequential chains should be used when each stage genuinely depends on the previous stage's output. If stages are independent, use a Parallel Chain instead for efficiency.

Quick Revision — Sequential Chain

- Sequential Chain = multiple prompt → model → parser stages connected end-to-end.
- The LLM can be invoked more than once inside a single chain.
- Built by extending the pipe chain: `prompt1 | model | parser | prompt2 | model | parser`.
- Each stage's parsed string output automatically becomes the next stage's prompt input.
- Useful for tasks like `generate report → summarize report`

4. Parallel Chain

Definition

A Parallel Chain executes two or more independent chains at the same time (rather than one after another), then optionally merges their outputs using an additional chain.

Why It Exists

Some tasks can be split into independent sub-tasks that don't depend on each other's output. For example, generating *notes* from a document and generating a *quiz* from the same document are independent operations — running them in parallel saves time compared to running them sequentially.

How It Works

1. `RunnableParallel` is used to define multiple chains that all receive the same input, keyed by name (e.g. "notes" and "quiz").
2. Each named chain internally is its own prompt → model → parser sequence.
3. The parallel chain returns a dictionary: `{"notes": ..., "quiz": ...}`.
4. A subsequent **merge chain** (a normal sequential chain) takes this dictionary as input and combines both outputs into one final document.
5. The parallel chain and merge chain are connected: `chain = parallel_chain | sequential_chain`.

Figure 6.3 shows the two branches executing simultaneously before being merged.

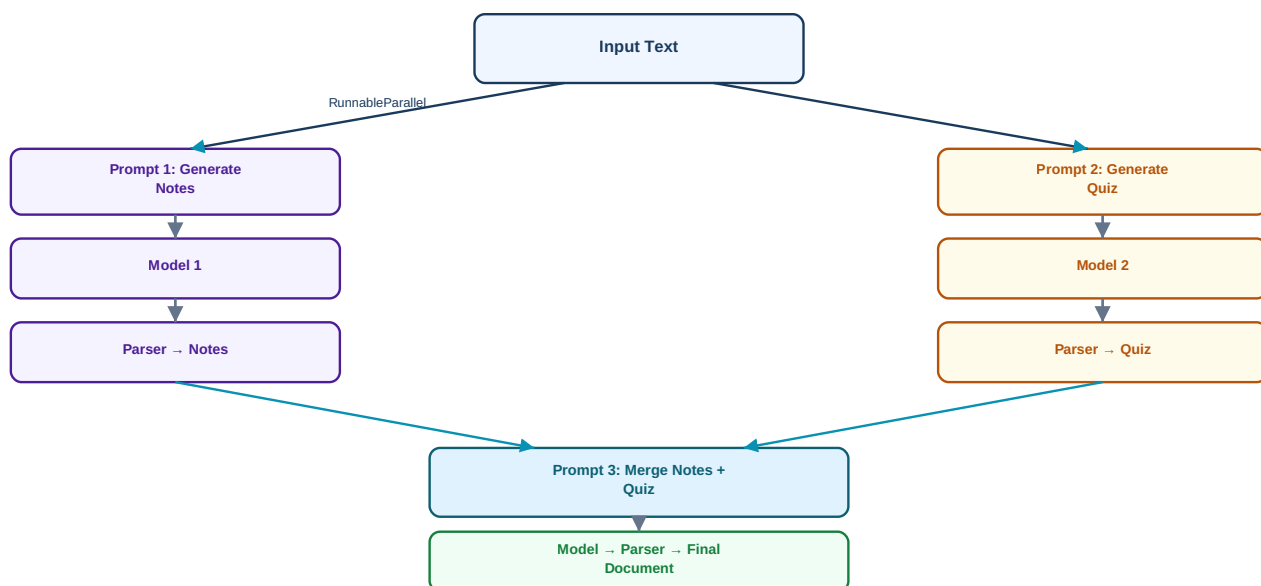


Figure 6.3 — Parallel Chain: independent notes/quiz branches merged into one document.

Example

```

from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from langchain_groq import ChatGroq
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableParallel
from dotenv import load_dotenv

load_dotenv()

llm = HuggingFaceEndpoint(
    repo_id = "Qwen/Qwen2.5-7B-Instruct"
)

# Model 1: Hugging Face-hosted Qwen model
model1 = ChatHuggingFace(llm = llm)

# Model 2: a different provider (Groq) used for the parallel branch
model2 = ChatGroq(
    model = "llama-3.3-70b-versatile"
)

text = """Support vector machines (SVMs) are a set of supervised learning methods..."""

# Branch 1: generate short notes from the text
prompt1 = PromptTemplate(
    template = "Read the document \n {text} \n and generate short notes out of the document.",
    input_variables = ["text"]
)

# Branch 2: generate a practice quiz from the same text
prompt2 = PromptTemplate(
    template = "Read the document \n {text} \n and generate 5 short practice quiz ques-ans out of it.",
    input_variables = ["text"]
)

# Merge stage: combine notes + quiz into one document
prompt3 = PromptTemplate(
    template = "Merge the notes : \n {notes} \n and the quiz questions : \n {quiz} \n together in a single document.",
    input_variables = ["notes", "quiz"]
)

parser = StrOutputParser()

# Both branches run in parallel, keyed by name; both receive the same input
parallel_chain = RunnableParallel({
    "notes" : prompt1 | model1 | parser,
    "quiz" : prompt2 | model2 | parser
})

# Sequential merge chain consumes the parallel chain's dict output

```

```

sequential_chain = prompt3 | model1 | parser

# Final chain: parallel branches feed directly into the merge chain
chain = parallel_chain | sequential_chain

result = chain.invoke({"text" : text})

chain.get_graph().print_ascii()

```

`RunnableParallel` takes a dictionary of named chains. Each key runs its own independent prompt → model → parser pipeline, but both receive the same input simultaneously, and the result is a dictionary that automatically satisfies `prompt3`'s two input variables. Two different model providers (`ChatHuggingFace` and `ChatGroq`) are deliberately used here to show that each parallel branch can use a completely different model.

COMMON MISTAKE

A frequent error is using an incorrect model name string for a provider (e.g. mismatching a provider's actual supported model identifiers). Always verify the exact model name from the provider's documentation before running the chain — an invalid model name will cause the entire chain to fail at runtime.

Quick Revision — Parallel Chain

- Parallel Chain runs two or more independent chains simultaneously using `RunnableParallel`.
- Each branch is keyed by a name (e.g. `\u201cnotes\u201d`, `\u201cquiz\u201d`) and returns a dictionary of results.
- Different branches can use entirely different models/providers.
- A subsequent chain can consume the parallel chain's dictionary output to merge results.
- Combined chain syntax: `chain = parallel_chain | sequential_chain`.

5. Conditional Chain

Definition

A Conditional Chain executes exactly one of multiple possible chains, based on a condition — behaving like an if-else statement inside LangChain's pipeline model. This differs from Parallel Chains, where all branches always execute.

Why It Exists

Some applications need to branch their logic based on a classification result. For example: classify user feedback as Positive or Negative, then generate a different type of reply depending on which sentiment was detected — only one reply-generation path should ever run.

How It Works

1. **Classification stage:** a prompt asks the LLM to classify feedback as "Positive" or "Negative".
2. **Structured output enforcement:** since raw LLM text output isn't guaranteed to be consistent, a `PydanticOutputParser` with a `Literal["Positive", "Negative"]` field is used to force the LLM to always return one of exactly two consistent values.
3. **Branching stage:** `RunnableBranch` defines multiple `(condition, chain)` tuples — if sentiment is Positive, run the positive-reply chain; if Negative, run the negative-reply chain; otherwise, run a default fallback wrapped via `RunnableLambda`.
4. **Final chain:** `classifier_chain | branch_chain` — the classification output feeds directly into the branch chain.

Figure 6.4 traces the classification and branching logic end-to-end.

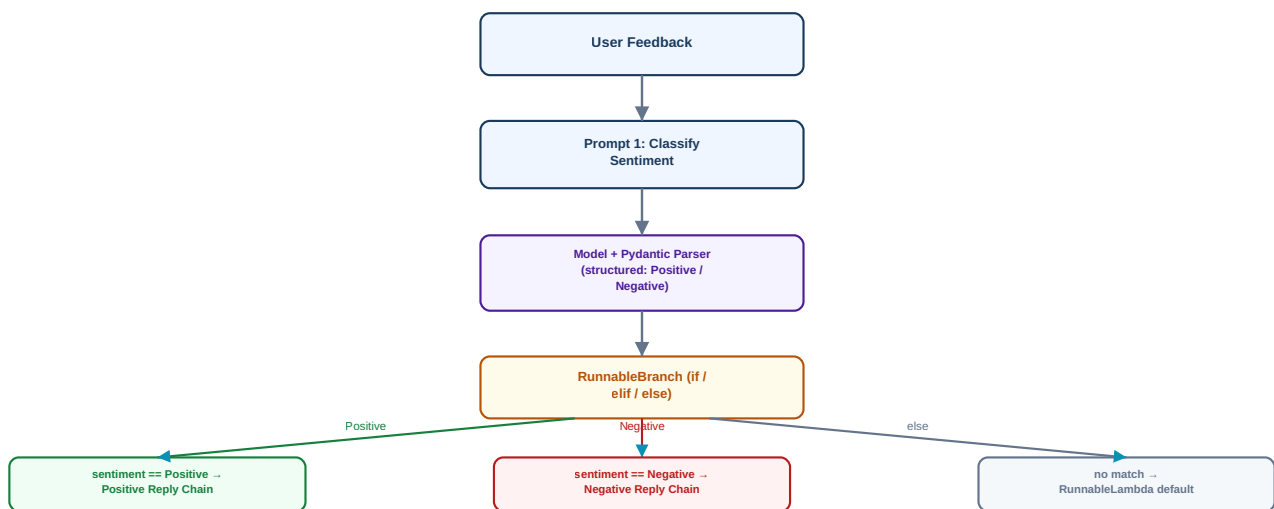


Figure 6.4 — Conditional Chain: classification feeding a `RunnableBranch` (if / elif / else).

Example

```

from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser, PydanticOutputParser
from langchain_core.runnables import RunnableBranch, RunnableLambda
from pydantic import BaseModel
from typing import Literal
from dotenv import load_dotenv

load_dotenv()

llm = HuggingFaceEndpoint(
    repo_id = "Qwen/Qwen2.5-7B-Instruct"
)

model = ChatHuggingFace(llm = llm)

parser1 = StrOutputParser()

# Structured schema forces the model to return ONLY "Positive" or "Negative"
class Feedback(BaseModel):
    sentiment : Literal["Positive","Negative"]

parser2 = PydanticOutputParser(pydantic_object = Feedback)

# Classification prompt – includes format instructions from the Pydantic parser
prompt1 = PromptTemplate(
    template = "Classify the Sentiment of the following feedback as Positive or Negative : \n {feedback} \n {format_instruction}.",
    input_variables = ["feedback"],
    partial_variables = {"format_instruction" : parser2.get_format_instructions()}
)

# Reply prompt for positive feedback
prompt2 = PromptTemplate(
    template = "Return an appropriate reply to this positive Review : {feedback}.",
    input_variables = ["feedback"]
)

# Reply prompt for negative feedback
prompt3 = PromptTemplate(
    template = "Write and appropriate reply to this negative Review : {feedback}.",
    input_variables = ["feedback"]
)

# Classification chain: outputs a structured Feedback object
classifier_chain = prompt1 | model | parser2

# Branching logic: exactly one path executes
branch_chain = RunnableBranch(
    (lambda x:x.sentiment == "Positive", prompt2 | model | parser1),
    (lambda x:x.sentiment == "Negative", prompt3 | model | parser1),
    RunnableLambda(lambda x: "Could not find sentiment.") # default fallback
)

```

```
# Full chain: classify first, then branch
final_chain = classifier_chain | branch_chain

result = final_chain.invoke({"feedback" : "This phone is terrible."})

final_chain.get_graph().print_ascii()
```

`Literal["Positive", "Negative"]` inside the Feedback Pydantic model constrains the LLM's structured output to exactly one of two allowed string values. `RunnableBranch` takes a series of `(condition_function, chain)` tuples, each checking `x.sentiment` on the classifier's output object; the last argument is the default chain, executed only if no earlier condition matches. `RunnableLambda` wraps a plain Python lambda so it can be treated as a `Runnable` and placed inside a chain — a raw lambda cannot be piped or invoked like a chain by itself.

IMPORTANT

In a Conditional Chain, only ONE branch executes per invocation. This is fundamentally different from a Parallel Chain, where ALL branches execute simultaneously every time.

COMMON MISTAKE

Relying on raw string output (`StrOutputParser`) for a classification step that feeds into conditional logic is risky — the LLM might return inconsistent phrasing (e.g. `\u201cThis is negative\u201d` instead of just `\u201cNegative\u201d`), breaking the condition check. Always use a structured output parser (like `PydanticOutputParser` with a `Literal` type) for any output that downstream logic depends on.

INTERVIEW TIP

Be prepared to explain why `RunnableLambda` is required to use a plain Python function/lambda inside a LangChain pipeline — it adapts a non-Runnable callable into the `Runnable` interface so it can be composed with `|` and invoked like any other chain component.

Quick Revision — Conditional Chain

- Conditional Chain executes exactly one branch based on a condition — like an if-else.
- Built using `RunnableBranch`, which takes `(condition, chain)` tuples plus a mandatory default chain.
- Structured output (`PydanticOutputParser` + `Literal`) should be used for any classification step feeding into a condition.
- Plain Python lambdas must be wrapped in `RunnableLambda` to be usable inside a chain.
- Full logic: `final_chain = classifier_chain | branch_chain`.

K. Key Takeaways

- **Chains** connect multiple LLM-application steps (prompt, model, parser, etc.) into a single automated pipeline using the pipe operator (`|`), part of LangChain Expression Language (LCEL).
- **Simple Chain**: Prompt → Model → Parser, a single straight-line flow.
- **Sequential Chain**: multiple prompt → model → parser stages chained together, where the LLM may be called more than once and each stage depends on the previous stage's output.
- **Parallel Chain**: uses `RunnableParallel` to run multiple independent chains simultaneously (each possibly using a different model/provider), returning a dictionary of named outputs that can be merged by a subsequent chain.
- **Conditional Chain**: uses `RunnableBranch` to execute exactly one chain out of several, based on a condition evaluated against structured (Pydantic) output.
- **RunnableLambda** is required to convert a plain Python function/lambda into a `Runnable` usable inside a chain.
- `chain.get_graph().print_ascii()` visualizes any chain's internal structure for debugging/understanding.
- The concept of **Runnable** (the underlying abstraction behind all chains, `RunnableParallel`, `RunnableBranch`, and `RunnableLambda`) is explored in full detail in the next chapter.

R. Revision Sheet

Chain

A pipeline connecting multiple LLM-application steps using the pipe operator, so each step's output automatically feeds the next step's input.

LCEL

LangChain Expression Language — the declarative syntax (using `|`) for composing chains.

Simple Chain

The minimal chain: Prompt → Model → Parser.

Sequential Chain

Multiple prompt → model → parser stages linked end-to-end; the LLM may be invoked more than once, with each stage depending on the prior stage's output.

Parallel Chain

Multiple independent chains executed simultaneously via `RunnableParallel`, returning a dict of named results that can feed a merge chain.

Conditional Chain

Exactly one chain executes based on a condition, built via `RunnableBranch` with `(condition, chain)` tuples and a mandatory default chain.

RunnableLambda

Wraps a plain Python function/lambda so it can be used as a Runnable inside a chain (needed for default/fallback branches).

PydanticOutputParser + Literal

Forces the LLM to return one of a fixed set of consistent values — essential when downstream conditional logic depends on the output.

I. Interview Questions

- Q1.** What is a Chain in LangChain, and what problem does it solve compared to manually invoking each component?
- Q2.** What is LCEL (LangChain Expression Language), and what operator is central to it?
- Q3.** How does a Sequential Chain differ from a Simple Chain?
- Q4.** When would you choose a Parallel Chain over a Sequential Chain?
- Q5.** What does `RunnableParallel` return, and how is that result typically consumed by a subsequent chain?
- Q6.** How does a Conditional Chain differ fundamentally from a Parallel Chain in terms of execution?
- Q7.** Why is structured output (e.g. via `PydanticOutputParser` with a `Literal` type) important when building a Conditional Chain?
- Q8.** What is the purpose of `RunnableLambda`, and why can't a plain Python lambda be used directly inside a chain?
- Q9.** What does `RunnableBranch`'s last argument represent, and why is it mandatory?
- Q10.** How can you visualize the internal structure of any LangChain chain?

F. Further Reading

- [LangChain official documentation on LangChain Expression Language \(LCEL\)](#)
- [LangChain documentation on `RunnableParallel`, `RunnableBranch`, and `RunnableLambda`](#)
- [LangChain documentation on `PydanticOutputParser` and structured outputs](#)
- [langchain_huggingface integration documentation \(`HuggingFaceEndpoint`, `ChatHuggingFace`\)](#)
- [langchain_groq integration documentation \(`ChatGroq`\)](#)